Internship Project Report

Java Full Stack Development Internship

Name	Adarsh Kanaparthi
Role	Java Full Stack Developer Intern
Organisation	Zaalima Development
Projects	1. High-Frequency Trading Matching Engine 2. Distributed Real-Time Order & Inventory Tracking System 3. AI-Powered Codebase & Documentation Assistant (RAG)
Source code	github.com/kanaparthiAdarsh-web/Zaalime_Java_Full_Stack_projects

Contents

Section	
1. Internship overview	Goals, tools and how the work was approached
2. Project 1 – HFT Matching Engine	Low-latency order matching in core Java
3. Project 2 – Order & Inventory System	Flash-sale backend: Redis, RabbitMQ, PostgreSQL, WebSockets
4. Project 3 – AI Codebase Assistant	Hybrid search (pgvector + Lucene) with cited answers
5. Testing and results	How each project was verified, with measured numbers
6. Learning outcomes	Skills gained
7. Conclusion	Summary and next steps

1. Internship overview

During this internship I built three backend projects of increasing size: a low-latency trading engine in plain Java, a distributed e-commerce backend made of several services, and an AI assistant that answers questions about a codebase. Each project started from a problem statement given by the company. I treated the suggested steps as a starting point and documented every place where I chose a different design, and why.

Way of working

- Build the core feature first, then test it under realistic pressure (concurrency, load, failures).
- When a test failed, find the real cause (logs, thread dumps, measurements) before changing code.
- Keep every number in the documentation measured, not estimated.
- Commit in small steps with clear messages; every project has a README with run instructions.

Main tools

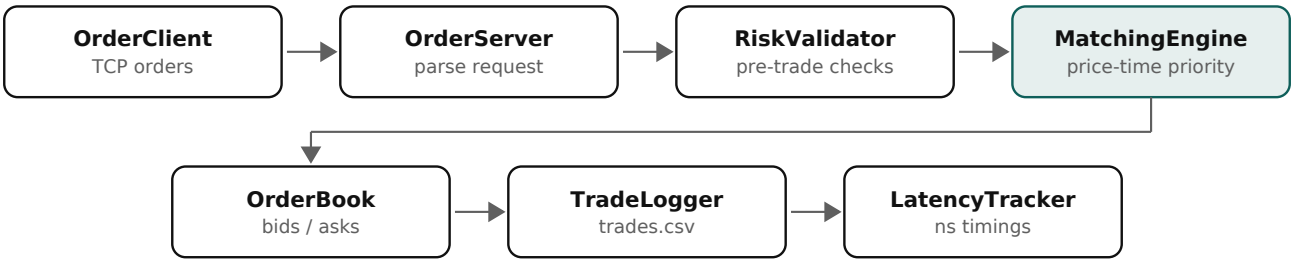
Area	Tools
Languages / frameworks	Java 21, Spring Boot 3.5, Spring Cloud Gateway, JavaScript
Data	PostgreSQL, pgvector, Redis, Apache Lucene
Messaging / real-time	RabbitMQ, WebSockets (STOMP), TCP sockets
AI	Local embeddings (bge-small, ONNX), Claude API (Anthropic Java SDK), Ollama
Quality	JUnit 5, Testcontainers, k6 load testing, Playwright (UI checks)
Delivery	Docker, Docker Compose, Git / GitHub

2. Project 1 – High-Frequency Trading Matching Engine

Problem. A stock exchange keeps an order book of buy and sell orders and matches them. In high-frequency trading this must happen in microseconds, and in Java the garbage collector can pause the program when many short-lived objects are created.

What I built. A limit order book and matching engine in plain Java, with a TCP server that accepts orders, pre-trade risk checks, nanosecond latency tracking and a CSV audit log of every trade.

Architecture



How it works

- The order book keeps two `TreeMap`s: bids sorted highest price first, asks lowest first, so the best price is always the first entry.
- An incoming buy order is matched against the cheapest asks while its price is high enough; whatever is left rests in the book (and the same in reverse for sells). Within a price level, older orders fill first – **price-time priority**.
- Orders come from a pre-allocated pool (`acquireOrder` / `releaseOrder`) and a single reusable Trade object is passed to listeners, so the hot path creates almost no garbage.
- `RiskValidator` rejects invalid ids, missing side, zero or oversized quantities and prices outside the allowed band before an order reaches the book.

Challenges and how I solved them

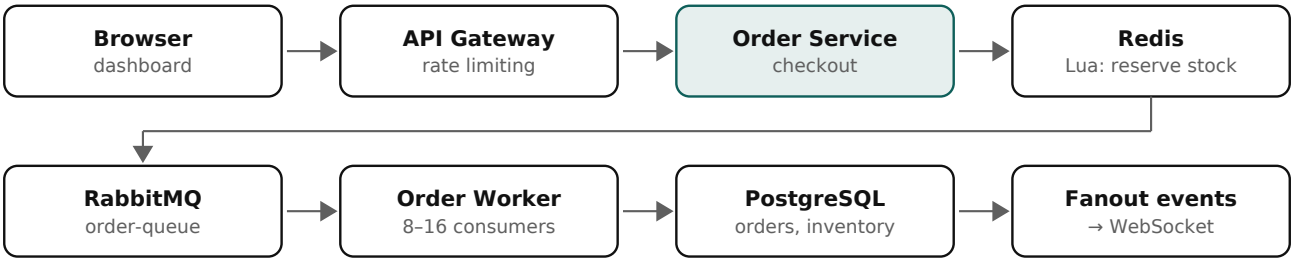
Challenge	Solution	Benefit
GC pauses from creating objects for every order and trade	Object pool for orders, one reusable Trade object	Stable, low latency on the hot path
Finding the best price quickly	Sorted TreeMaps per side; best price = first entry	O(log n) insert, instant best-price lookup
Bad orders reaching the book	Pre-trade RiskValidator	Unsafe orders are rejected early
Knowing whether it is actually fast	LatencyTracker measuring each order in nanoseconds	Speed is measured, not guessed

3. Project 2 – Distributed Real-Time Order & Inventory System

Problem. In a flash sale thousands of users try to buy a few items at the same moment. A normal database-centred design oversells (two buyers get the last item), slows down under load, and charges customers twice when a request is retried.

What I built. Four Spring Boot modules – an API gateway, an order service, a background worker and a shared module – with Redis, RabbitMQ and PostgreSQL, plus a live dashboard.

Architecture

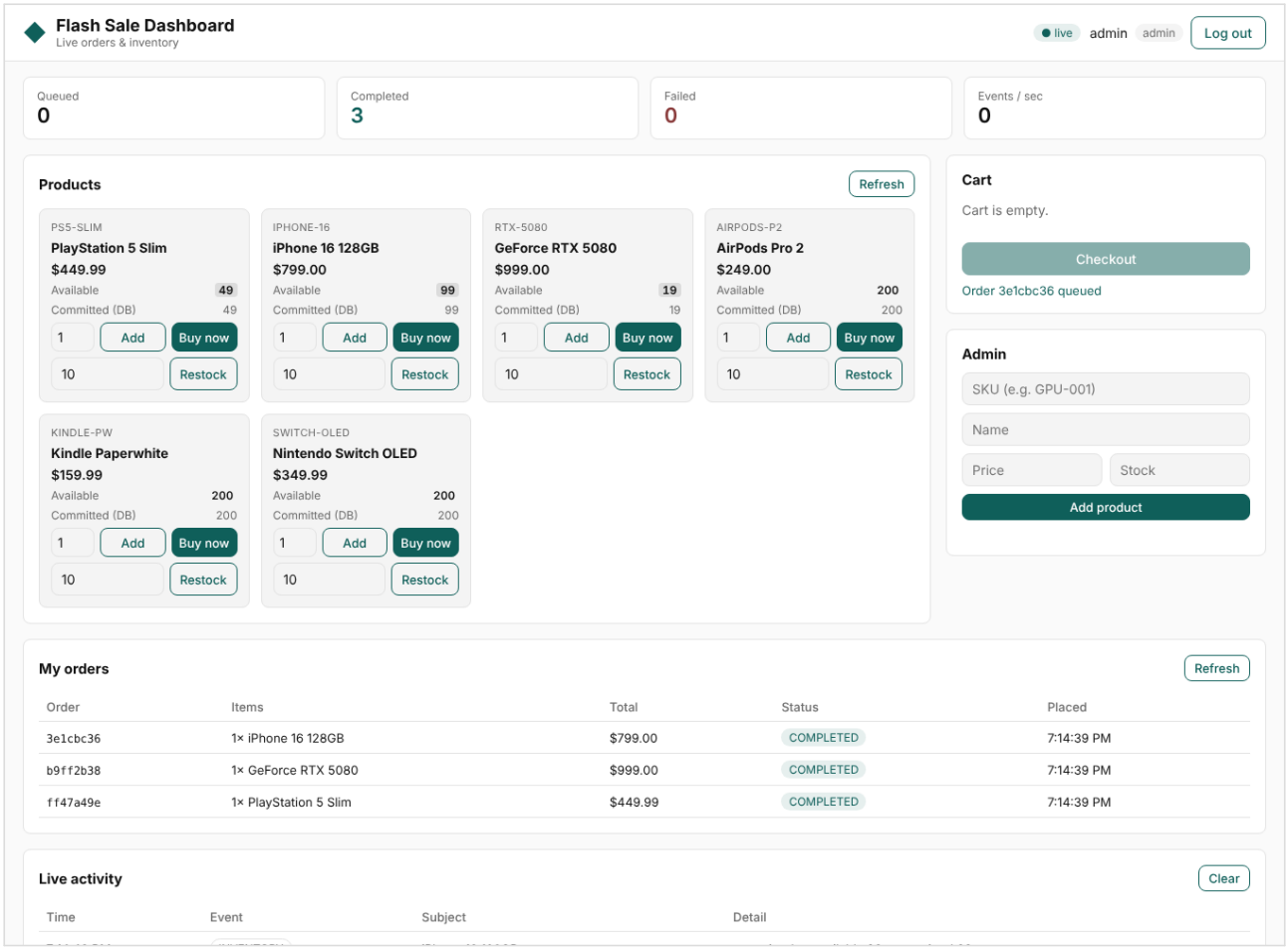


Checkout flow

- **Idempotency:** the request's Idempotency-Key is claimed in Redis; a retry gets the original answer instead of a second order.
- **Atomic reservation:** one Lua script checks every item and then decrements every item. Redis runs scripts one at a time, so overselling is impossible – no locks needed.
- **Queue:** the order is published to RabbitMQ with a publisher confirm; the API answers 202 Accepted without touching PostgreSQL.
- **Commit:** the worker writes the order and runs a conditional stock update in one transaction; a CHECK (quantity >= 0) constraint is the last safety net.
- **Live updates:** each step (PENDING → PROCESSING → COMPLETED / FAILED) is broadcast over a fanout exchange and pushed to browsers over WebSockets.

Challenges and how I solved them

Challenge	Solution	Benefit
Overselling under concurrency	Redis Lua check-and-decrement; DB CHECK constraint as backup	Exactly the available stock is sold
Database overload during spikes	RabbitMQ queue + background worker	Checkout never waits for PostgreSQL
Double orders on network retries	Idempotency keys in Redis and a UNIQUE constraint in PostgreSQL	A retry returns the same order
1,000-user load test hung ~40% of requests	Thread dumps showed virtual threads pinned by the RabbitMQ client (Java 21) → deadlock. Switched to a sized platform thread pool and async publisher confirms	0 errors at 1,000 concurrent checkouts
Database outage	Circuit breakers (Resilience4j); catalog and order status fall back to Redis	Orders still accepted and completed after recovery



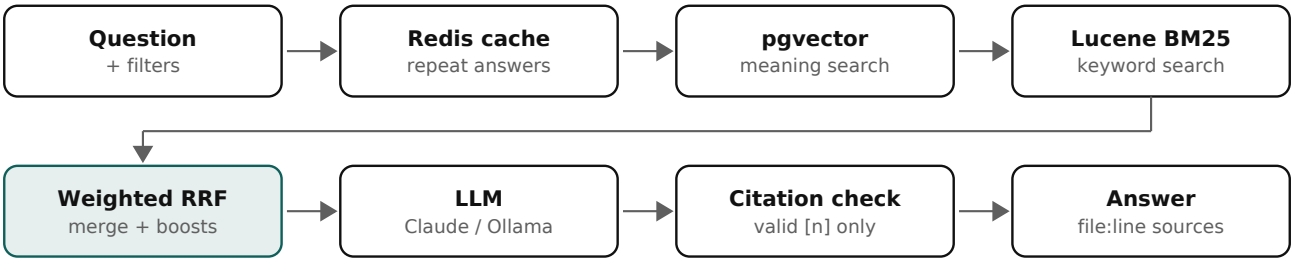
Live dashboard: products with live and committed stock, cart, orders and the real-time activity log

4. Project 3 – AI-Powered Codebase & Documentation Assistant (RAG)

Problem. Developers lose time searching big codebases. Keyword search misses questions asked in plain English, while AI search understands meaning but often misses exact function names.

What I built. A Spring Boot service that indexes Java, Python, Markdown and PDF files and answers questions with the exact files and line numbers the answer came from.

Architecture

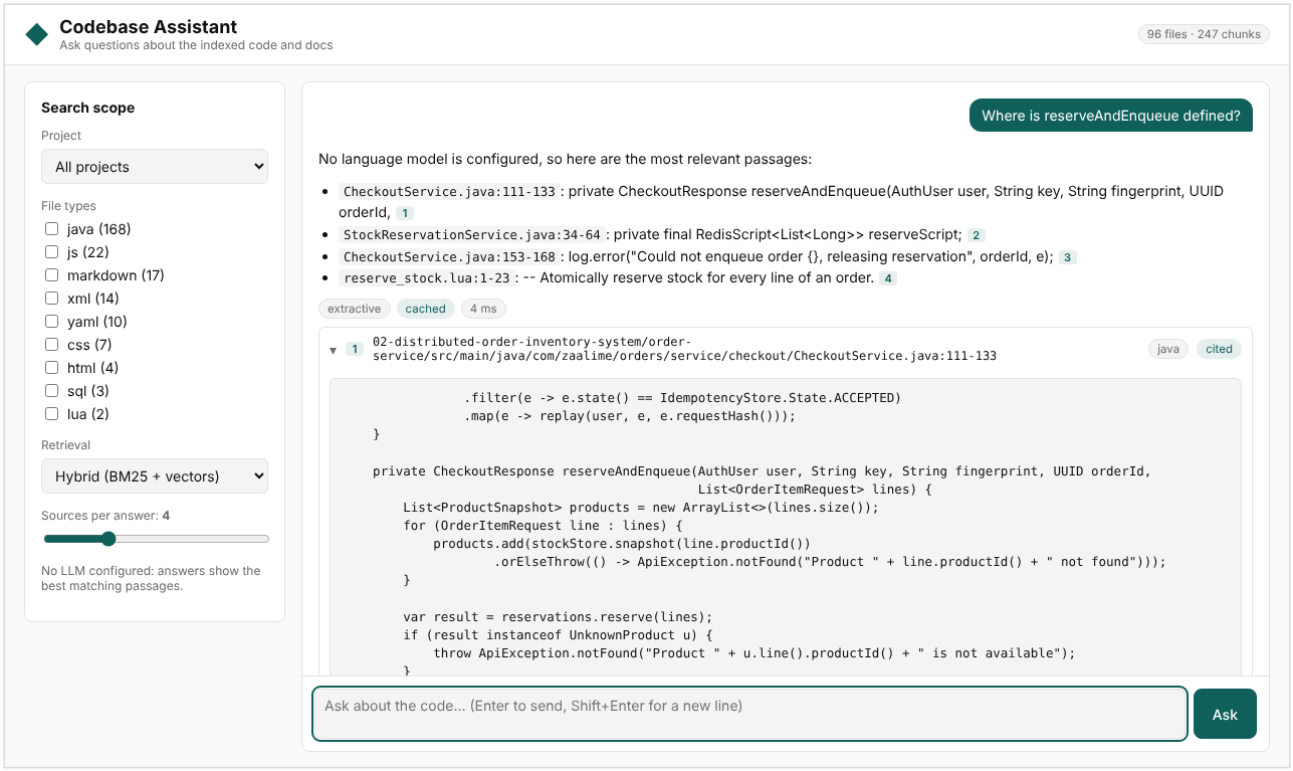


How it works

- **Ingestion:** files are split into chunks of whole lines, cut at class, method or heading boundaries, so every citation has an exact line range. Unchanged files are skipped on re-indexing.
- **Embeddings:** the bge-small model runs inside the JVM (ONNX), so indexing needs no API key and no code leaves the machine.
- **Hybrid retrieval:** pgvector (semantic) and Lucene BM25 (exact words, with camelCase and snake_case split) each return 20 candidates, merged with weighted Reciprocal Rank Fusion.
- **Answers:** Claude (official Anthropic Java SDK) answers only from the numbered sources and must cite them; without an API key the service returns the best matching lines instead.

Challenges and how I solved them

Challenge	Solution	Benefit
AI search misses exact function names	Hybrid search: pgvector + Lucene BM25, merged with RRF	Finds both ideas and exact names
Plain fusion made results worse for English questions	Weighted RRF: keyword list counts less for prose, more when the question names an identifier	Better ranking for both question types
Files that mention a function ranked above the file that defines it	Definition boost for chunks that declare the identifier	The defining file ranks first
One long README filled every result slot	Maximum 2 chunks per file	Vector hit rate 86% → 91% (first eval)
No API key / LLM outage	Extractive fallback answer with a warning	Always returns a useful answer



Chat UI: the defining method is the first result, with clickable citations and the exact line range

5. Testing and results

Every number below comes from a test or measurement run on the finished code.

Project 2 – Order & Inventory System

Test	Result
k6: 1,000 buyers checking out at the same instant for 500 units	Exactly 500 accepted, 500 sold out, 0 errors; Redis and PostgreSQL both end at 0
1,000 concurrent reservations of 100 units (integration test)	Exactly 100 succeed
Same request retried with the same Idempotency-Key	Same order returned, nothing charged twice
PostgreSQL stopped during live traffic	Catalog served from Redis, checkout accepted, order completed ~4 s after recovery
60 rapid logins from one IP	40 allowed, 20 rejected with HTTP 429
Automated tests	18 integration tests (real PostgreSQL, Redis, RabbitMQ)

Project 3 – AI Codebase Assistant (30 evaluation questions)

Index	Search	Right file in top 4	MRR	Code-name MRR
Projects 1 + 2	Hybrid	93%	0.79	0.95
Projects 1 + 2	Vector only	93%	0.81	0.94
Projects 1 + 2	Keyword only	83%	0.70	0.95
+ Project 3 (noisy)	Hybrid	93%	0.77	0.95
+ Project 3 (noisy)	Vector only	90%	0.72	0.79

On a clean codebase vector and hybrid search perform the same. When other files also mention the same names, vector-only search drops (code-name MRR 0.94 → 0.79) while hybrid stays at 0.95 – the reason hybrid is the default. Other results: indexing 96 files takes ~12 s, a question ~30 ms after warm-up, repeated questions come from Redis, and 36 automated tests pass (including integration tests on real pgvector).

6. Learning outcomes

- **Backend engineering:** Spring Boot services, REST API design, validation, JWT security, Flyway migrations.
- **Concurrency and distributed systems:** atomic operations, message queues, idempotency, circuit breakers, and how thread pools and locks behave under load.
- **Data:** PostgreSQL constraints and transactions, Redis, vector search with pgvector, full-text search with Lucene.
- **AI engineering:** embeddings, retrieval-augmented generation, prompt design against hallucination, and evaluating retrieval with hit rate and MRR.
- **Quality:** integration tests with Testcontainers, load testing with k6, reading thread dumps to find the real cause of a failure.
- **Delivery:** Docker Compose setups, clear READMEs and small, well-described commits.

7. Conclusion

The three projects moved from a single-process, latency-focused Java program to a distributed system and finally to an AI service. The most valuable lesson was that a design is only proven when it is tested under pressure: the load test in Project 2 and the evaluation in Project 3 both exposed problems that normal use would not have shown, and fixing them made the systems genuinely reliable.

Next steps: stream AI answers to the browser, run the Project 3 answers against the live Claude API, and deploy Project 2 with multiple instances behind the gateway.

Source code: https://github.com/kanaparthiAdarsh-web/Zaalime_Java_Full_Stack_projects